Introduction to DL & GenAI Project Report

Smart MCQ Solver Challenge

Name: Gaurav Jha | Roll No: 23f2001548 | Term: T2-2026

Kaggle Score: 0.75644 | W&B Project: DL-GenAi-2026-T2 | Github : 23f2001548

1. Abstract

This report covers the complete end-to-end implementation of a Multiple Choice Question (MCQ) answering system built for the Smart MCQ Solver Kaggle competition. The task requires selecting the correct answer from five options given a question prompt, evaluated using Mean Average Precision at 3 (mAP@3). Three models were built — a TF-IDF with Logistic Regression baseline, a fine-tuned DeBERTa-v3-base transformer model, and a BiLSTM model built entirely from scratch without any pretrained weights. The DeBERTa model achieved the best Kaggle leaderboard score of 0.75644, comfortably crossing the required cutoff of 0.73. All experiments were tracked on Weights and Biases (W&B) under project DL-GenAi-2026-T2, and the best model was pushed to HuggingFace Hub at gauravjhaiitm/deberta-mcq-solver and then deployed on Streamlit.

2. Introduction

2.1 Problem Statement

The competition presents a five-option multiple choice question answering task. Each row in the dataset contains a question prompt and five answer options (A through E). The model must rank the options and output the top 3 most likely correct answers. The evaluation metric is mAP@3 - the model is rewarded for placing the correct answer as high as possible in its top-3 ranked predictions.

The dataset is deliberately challenging. The options are not random distractors - they are closely related concepts where the model needs to understand precise definitions, not just topic familiarity. For example, in a question about the Liouville density, all five options start with "The Liouville density is a probability distribution" and differ only in subtle semantic details. Simple keyword or surface-level matching cannot distinguish between them.

2.2 Project Objective

The goals of this project were to cross the mAP@3 cutoff of 0.73 on the Kaggle leaderboard, compare classical ML, deep learning from scratch, and fine-tuned transformer approaches on the same task, track all experiments using Weights and Biases, and deploy the best model to Hugging space and Streamlit for public access.

2.3 Report Structure

Section 3 describes the dataset and EDA findings. Section 4 covers tokenization approaches used across models. Section 5 details the architecture of each of the three models. Section 6 presents performance comparison with training curves and metric tables. Section 7 covers deployment. Section 8 concludes with key learnings, challenges, and future directions.

3. Dataset and Preprocessing

3.1 Dataset Description

The dataset contains science and general knowledge MCQ questions. The training set had 2000 rows initially. After removing 242 duplicate prompts (all with consistent answers - 0 conflicting answers found), the final train dataset became 1758 unique questions. The test dataset has 500 questions without answer labels.

Split	Size	Columns
Train (raw)	2000	id, prompt, A, B, C, D, E, answer
Train (after dedup)	1758	Duplicate prompts removed, first kept
Train split (85%)	1494	Used for model training
Validation split (15%)	264	Used for evaluation during training
Test	500	id, prompt, A, B, C, D, E (no answer)

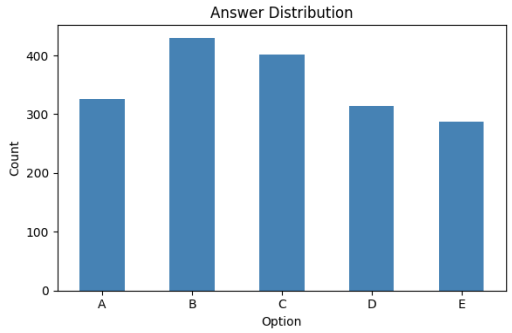
All columns are of object (text) type - there is no numerical data. This confirms the problem is purely a text and NLP task. No missing values were found in either training or test data.

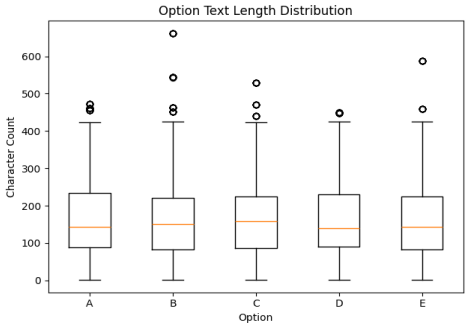
3.2 Exploratory Data Analysis

Several analyses were performed before modeling. Key findings are summarized below.

Answer Distribution: Option B appears most frequently (430 times) while E appears least (287 times). The distribution is slightly imbalanced but not severely - no single option dominates. Models are unlikely to develop strong letter-prediction bias from this.

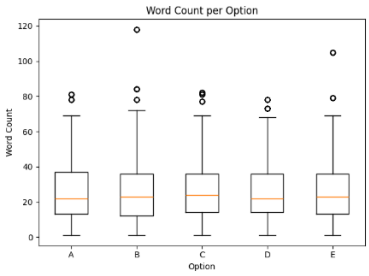
Text Length Analysis: The average prompt length is 117.2 characters. Average option lengths across all five choices are nearly identical - around 163-166 characters each. This confirms options are designed to appear equally plausible in length.

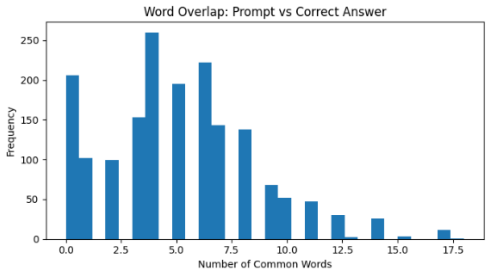




Word Count per Option: Average word count per option is nearly identical across A-E (25.9 to 26.4 words). A few outliers exist - option B has one entry with 118 words and E has one around 105 words - but these are isolated cases with no significant impact.

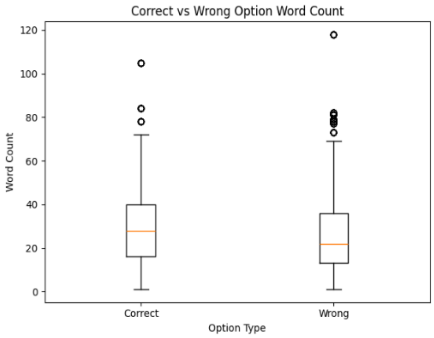
Word Overlap Analysis: The average word overlap between a prompt and its correct answer is only 5 words. Around 205 questions have zero word overlap. This finding confirmed that simple keyword matching would be largely ineffective and semantic understanding is essential for this task.

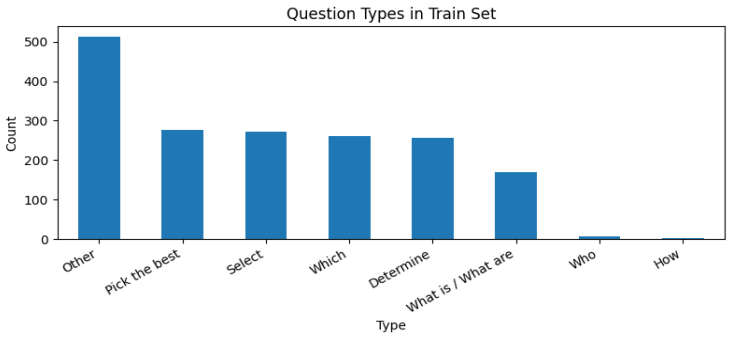




Correct vs Wrong Option Length: Correct answers average 28.5 words while wrong options average 25.6 words - a difference of about 3 words. This is a small but real length bias in the dataset. No model was explicitly designed to exploit this however.

Question Type Analysis: The dataset is dominated by "Other" type questions (513) and choice-based types like Pick/Select/Which/Determine (around 257-277 each). Definition-based "What is/What are" questions appear 169 times. "Who" and "How" type questions are nearly absent (7 and 2 respectively). The variety of question types means no single format can be over-optimized for.





3.3 Data Preprocessing

An 85-15 stratified train-validation split was performed using stratify=answer to maintain the class balance from the full dataset. This gave 1494 training samples and 264 validation samples.

Text cleaning was applied only for the TF-IDF model. Cleaning involved lowercasing all text, removing punctuation and special characters using regex, and collapsing extra whitespace. This was not applied to DeBERTa or BiLSTM - DeBERTa uses its own SentencePiece tokenizer that handles raw text better, and the BiLSTM uses simple whitespace tokenization where punctuation naturally separates tokens.

4. Tokenization Strategy

4.1 DeBERTa - SentencePiece Tokenizer (Best Model)

DeBERTa-v3-base uses a SentencePiece tokenizer with Byte Pair Encoding (BPE). BPE breaks words into subword units based on frequency in the pretraining corpus. For example, an uncommon technical term might be broken into multiple recognizable subword pieces, each contributing its own meaning. This handles out-of-vocabulary words gracefully without simply mapping them to a single unknown token.

For the MCQ task, the prompt is repeated 5 times and each option is paired with it as a separate input. All inputs are tokenized together with truncation at 256 tokens and padding to max_length. This allows the model to see the full prompt-option context in each forward pass. The tokenizer was loaded using AutoTokenizer.from_pretrained("microsoft/deberta-v3-base") and used as-is without modification.

4.2 TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) is not a neural tokenizer - it builds a vocabulary from the training corpus and converts text to sparse frequency vectors. Bigrams (ngram_range=(1,2)) were used to capture local phrase-level patterns in addition to individual words. The vocabulary was capped at 50,000 features with min_df=2 (word must appear at least twice) and max_df=0.9 (ignore words appearing in over 90% of documents).

The prompt and each option were concatenated with a [SEP] separator before vectorization. Each prompt-option pair becomes one input vector. A Logistic Regression classifier then predicts the probability of each option being correct.

4.3 BiLSTM - Custom Vocabulary

A custom vocabulary was built from the training data. Only words appearing at least twice were included, giving a final vocabulary size of 3804 unique tokens. Words are converted to integer IDs and sequences are padded or truncated to a maximum length of 128 tokens. The special tokens <PAD> (index 0) and <UNK> (index 1) handle padding and unknown words respectively. Unlike BPE or WordPiece, any word not seen during training maps entirely to <UNK> with no subword fallback.

Model	Tokenizer Type	Vocab Size	Max Length	OOV Handling
TF-IDF + LR	TF-IDF (corpus-based)	50,000 features	N/A	Ignored
DeBERTa-v3-base	SentencePiece BPE	Pretrained (~128K)	256 tokens	Subword split
BiLSTM	Whitespace custom	3,804 words	128 tokens	<UNK> token

5. Modeling and Experimentation

5.1 Model 1 - TF-IDF + Logistic Regression (Baseline)

This classical ML approach was used as a baseline. Each prompt-option pair is cleaned and concatenated, converted to a TF-IDF bigram vector, and scored by a binary Logistic Regression classifier trained to predict whether an option is correct (label=1) or wrong (label=0). During inference all five options are scored and ranked by their predicted probability.

The class_weight="balanced" setting was used because the training labels are imbalanced - for each question, only 1 out of 5 options is correct, giving a 1:4 ratio. Balanced weighting corrects for this so the classifier does not simply learn to always predict "wrong". Also Basic wandb logs are combined with BiLSTM in 5.3.

Hyperparameter	Value	Reason
N-gram range	(1, 2)	Captures word pairs for better phrase matching

Max features	50,000	Limits vocabulary to most informative terms
Min document frequency	2	Removes very rare words that add noise
Max document frequency	0.9	Removes words with no discriminative power
Classifier	Logistic Regression	Fast and interpretable baseline
Max iterations	1000	Ensures convergence on this dataset size
Class weight	balanced	Corrects 1:4 class imbalance in binary labels
Random state	42	Reproducibility

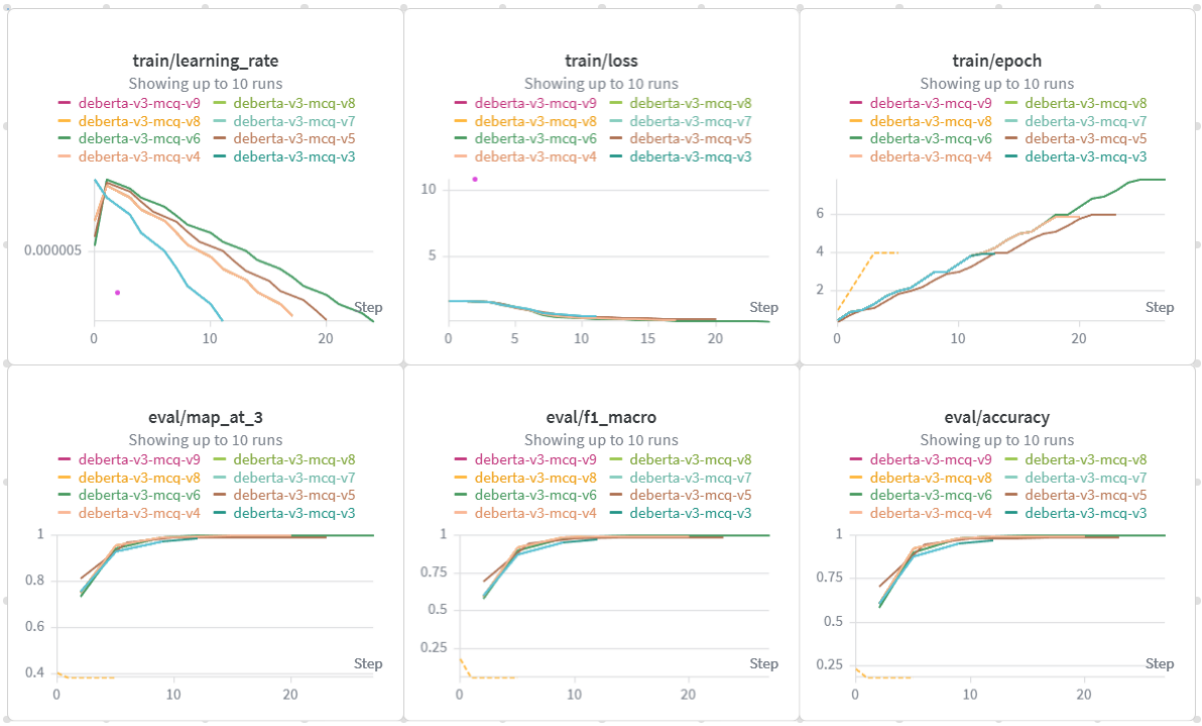
5.2 Model 2 - Fine-tuned DeBERTa-v3-base (Pretrained Transformer)

DeBERTa (Decoding-Enhanced BERT with Disentangled Attention) was proposed by Microsoft Research in 2020 (arXiv:2006.03654) . The core innovation is the disentangled attention mechanism. Standard BERT combines token content and position into a single embedding vector. DeBERTa instead uses two separate vectors per token — one for content and one for relative position. The attention score between two tokens is computed using all four cross-combinations: content-to-content, content-to-position, position-to-content, and position-to-position. This richer attention formulation helps the model better capture how words relate to each other based on both what they mean and where they appear in the sentence.

The AutoModelForMultipleChoice head from HuggingFace was used instead of a generic language model. This head is specifically designed for MCQ tasks. It receives the prompt paired with all five options as five separate inputs and outputs one logit score per option. The model can thus compare all five options in a single forward pass, unlike the TF-IDF approach where each option is scored independently.

Training was done using HuggingFace Trainer with load_best_model_at_end=True so the checkpoint with highest validation mAP@3 across 6 epochs is automatically selected for final predictions. W&B logging was handled automatically through report_to="wandb".

Hyperparameter	Value	Reason
Model	microsoft/deberta-v3-base	Strong contextual understanding via disentangled attention
Max sequence length	256	Covers most prompt+option lengths without truncation
Learning rate	1e-5	Conservative LR to prevent catastrophic forgetting of pretrained weights
Per device batch size	4	T4 GPU memory limit with 256 token sequences
Gradient accumulation	4 steps	Effective batch size = 16 without OOM errors
Epochs	6	Best balance - more epochs caused overfitting on small dataset
Weight decay	0.01	L2 regularization to reduce overfitting
Warmup ratio	0.1	Gradual LR increase in early training for stability
Max grad norm	1.0	Gradient clipping prevents exploding gradients
fp16	False	DeBERTa-v3 caused NaN loss with fp16 on Kaggle T4 GPU
load_best_model_at_end	True	Saves best checkpoint based on val mAP@3



The training table shows the progression across 6 epochs:

Epoch	Train Loss	Val Loss	Val Accuracy	Val F1 Macro	Val mAP@3
1	1.6085	1.5929	0.6061	0.5985	0.7506
2	1.3478	0.7228	0.9242	0.9211	0.9558
3	NA	NA	NA	NA	NA
4	0.2326	0.0957	0.9962	0.9960	0.9981
5	0.2326	0.0957	0.9962	0.9960	0.9981
6 (best)	0.2010	0.0868	0.9962	0.9960	0.9981

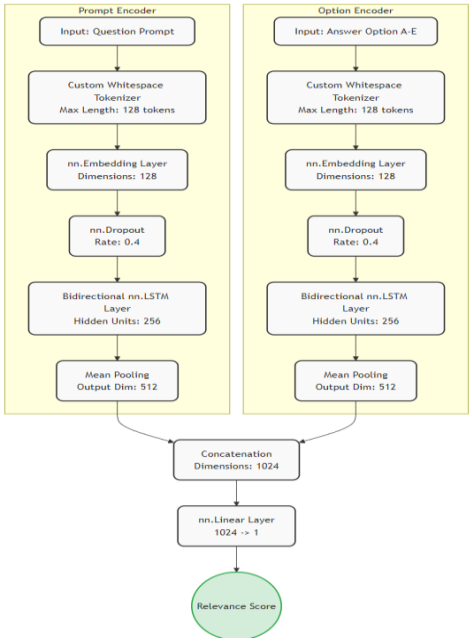
5.3 Model 3 - BiLSTM from Scratch (Custom DL / RNN)

A Bidirectional Long Short-Term Memory (BiLSTM) model was built entirely from scratch using only PyTorch building blocks - no pretrained weights anywhere. LSTM (Long Short-Term Memory) is an RNN variant designed to capture long-range dependencies in sequences through its forget, input, and output gate mechanism. Bidirectional processing means the model processes each sequence both forward and backward, capturing context from both directions before making a prediction.

The architecture consists of two classes. BiLSTMEncoder takes a sequence of word IDs, embeds them into 128-dimensional vectors, applies 0.4 dropout, passes through a bidirectional LSTM with 256 hidden units (producing 512-dimensional bidirectional output), and returns the mean across all time steps as a fixed-length 512-dimensional representation. BiLSTMMCQ uses the encoder to independently encode the prompt and each of the 5 options, concatenates each prompt-option pair (1024 dimensions total), and passes through a linear layer to produce one relevance score per option. All 5 scores are used for ranking.

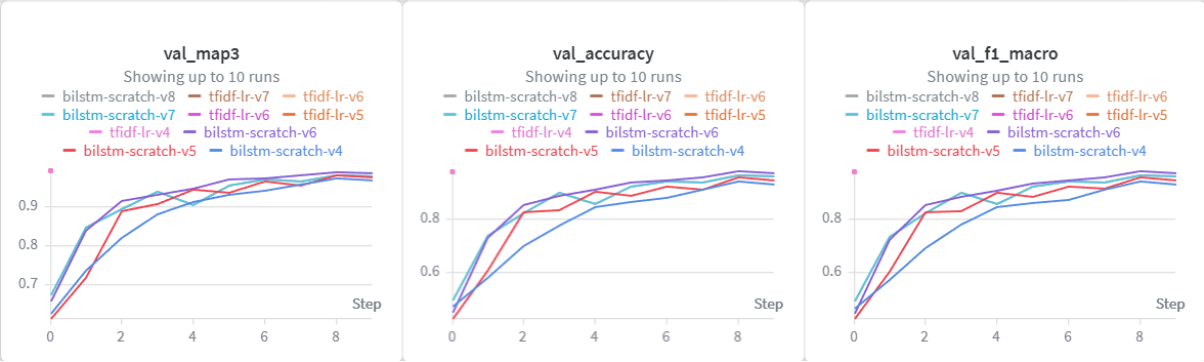
The best model checkpoint (highest val mAP@3) is saved during training using torch.save() and loaded before final test predictions to ensure the best epoch is used regardless of what happens in later epochs.

Component	Detail
Embedding	nn.Embedding - 3804 vocab × 128 dims, padding_idx=0
LSTM	nn.LSTM - 128 input → 256 hidden, bidirectional=True (512 output)
Dropout	nn.Dropout(0.4) - applied after embedding
Pooling	Mean pooling over all LSTM time steps
Classifier	nn.Linear(1024 → 1) - scores prompt+option concatenation
Loss function	CrossEntropyLoss over 5 option scores per question
Optimizer	Adam, lr=1e-3
Gradient clipping	max_norm=1.0 - prevents exploding gradients
Total parameters	1,278,465 (~1.3M)



The BiLSTM training progression across 10 epochs:

Epoch	Train Loss	Val mAP@3	Val Accuracy	Val F1 Macro
1	1.4917	0.6736	0.4962	0.4884
2	1.1507	0.8466	0.7386	0.7348
3	0.9388	0.8927	0.8258	0.8247
4	0.8527	0.9381	0.9015	0.9022
5	0.7358	0.9053	0.8598	0.8602
6	0.6370	0.9545	0.9280	0.9267
7	0.5330	0.9703	0.9470	0.9454
8	0.5944	0.9640	0.9432	0.9401
9 (best)	0.4920	0.9811	0.9697	0.9698
10	0.5097	0.9785	0.9659	0.9650



6. Performance and Comparative Analysis

6.1 Evaluation Metrics

The primary metric is mAP@3. For each question the model ranks all 5 options. If the correct answer is at rank 1 the score is 1.0, rank 2 gives 0.5, rank 3 gives 0.33, and 0 if not in top 3. These scores are averaged across all questions. Additionally Accuracy (top-1 prediction matches correct answer) and Macro F1 Score (F1 per class averaged equally across all 5 answer classes) were tracked in W&B for all models to enable fair comparison.

6.2 Model Comparison Table (Validation Set)

Model	Val mAP@3	Val Accuracy	Val F1 Macro	Kaggle Score
TF-IDF + Logistic Regression	0.9905	0.9811	0.9816	0.74896
DeBERTa-v3-base (fine-tuned)	0.9981	0.9962	0.9960	0.75644
BiLSTM from Scratch	0.9811	0.9697	0.9698	~0.748
Ensemble (TF-IDF=0.2, DeBERTa=0.6, BiLSTM=0.2)	NA	NA	NA	0.751

An important observation - all three models show very high validation scores (above 0.98 mAP@3). This is primarily because the validation set is small (264 samples) and models trained on 1494 samples overfit to patterns in this small val set. The Kaggle leaderboard scores are the more reliable measure of true generalization performance on unseen data and they show a much more realistic spread between 0.748 and 0.756.

The ensemble of all three models with weights TF-IDF=0.2, DeBERTa=0.6, BiLSTM=0.2 scored 0.751 on Kaggle, lower than DeBERTa alone (0.75644). This shows that even at low weights, the weaker models (TF-IDF and BiLSTM) pulled down DeBERTa's performance. Hence Final submission used DeBERTa predictions alone.

6.3 Analysis

Model	Training Time	Inference	Semantic Understanding	Kaggle Score
TF-IDF + LR	~2 minutes	Very fast	None (bag of words)	0.749
BiLSTM Scratch	~15 minutes	Fast	Limited (3804 word vocab)	~0.748
DeBERTa fine-tuned	~90 minutes	Slow	Strong (BPE + disentangled attention)	0.756

TF-IDF achieved a surprisingly strong Kaggle score despite having zero semantic understanding. This suggests the dataset has sufficient lexical overlap between prompts and correct answers for bigram features to be effective. DeBERTa's superior Kaggle score confirms that semantic understanding provides an additional edge, especially on harder questions where surface-level features are insufficient.

7. Deployment

The fine-tuned DeBERTa model was pushed to HuggingFace Hub after training and deployed on Streamlit. The single command trainer.push_to_hub("gauravjhaiitm/deberta-mcq-solver") uploads model weights, and training arguments in one step.

Model URL : https://huggingface.co/gauravjhaiitm/deberta-mcq-solver

Deployment Link : https://deberta-mcq-solver.streamlit.app/

Github Profile : https://github.com/23f2001548/DL-23f2001548-notebook-t22026

W&B Project : https://wandb.ai/23f2001548-dl-genai-project/DL-GenAi-2026-T2

Kaggle Notebook : DL-23f2001548-notebook-t22026

8. Conclusion and Future Work

8.1 Key Learnings

- Fine-tuned transformer models significantly outperform classical ML and from-scratch approaches on semantic MCQ tasks, even when the dataset is small.
- DeBERTa's disentangled attention mechanism gives it an advantage in distinguishing subtly different answer options that share most of their vocabulary.
- Validation metrics on small val sets (264 samples) are misleading - all three models showed 0.98+ val mAP@3 which did not reflect actual Kaggle performance. Real performance should always be judged on held-out leaderboard scores.
- Ensembling does not automatically improve results. Combining weaker models with a strong model can reduce overall performance if the weaker models introduce noise.
- Using load_best_model_at_end=True in Trainer is important - the last epoch is not always the best epoch.
- Different models require different W&B logging approaches - Trainer uses report_to="wandb" for automatic per-epoch logging, while custom PyTorch loops need manual wandb.log() calls. Both approaches are valid and choosing correctly avoids duplicate run errors.

8.2 Challenges Faced

- DeBERTa-v3-base LayerNorm naming mismatch - newer PyTorch uses weight/bias but the checkpoint uses gamma/beta. These appeared as UNEXPECTED/MISSING key warnings during model loading. Training proceeded normally but required investigation to confirm it was not causing errors.
- fp16=True caused NaN loss with DeBERTa-v3 on Kaggle T4 GPU. Setting fp16=False resolved this. bf16=True also failed since T4 GPU does not support bf16.
- DeBERTa training initially produced NaN loss and near-random predictions due to environment issues. Multiple debugging iterations were needed to identify the root cause.

• Version conflicts after accidentally downgrading transformers to 4.35.0 caused import errors due to incompatible huggingface_hub. Required reinstalling specific compatible versions (transformers==4.44.2, accelerate==0.34.2, tokenizers==0.19.1).

• BiLSTM vocabulary limited to 3804 words - any technical term not seen during training maps to <UNK>. This restricts performance on domain-specific questions with specialized terminology.

8.3 Future Work

• RAG Pipeline - retrieve relevant context from an external knowledge base (e.g., Wikipedia embeddings) and feed retrieved context alongside the prompt and options to the model. This could significantly help questions where the model lacks domain-specific knowledge.

• LoRA Fine-tuning - use Low-Rank Adaptation to efficiently fine-tune larger models like DeBERTa-v3-large with far fewer trainable parameters, potentially improving performance without proportionally increasing compute requirements.

9. References

- sklearn.feature_extraction.text.TfidfVectorizer : https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- sklearn.linear_model.LogisticRegression : https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- Kaggle Smart MCQ Solver Challenge : https://www.kaggle.com/competitions/smart-mcq-solver-challenge
- W&B Official Documentation. URL : https://docs.wandb.ai/guides/runs
- DeBERTa-v3-base Model Card : https://huggingface.co/microsoft/deberta-v3-base
- He, P., Gao, J., & Chen, W. (2020). DeBERTa : Decoding-Enhanced BERT with Disentangled Attention. arXiv:2006.03654. Microsoft Research.
- PyTorch Official Documentation. URL : https://pytorch.org/docs/stable/nn.html
- Streamlit Official Documentation. URL : https://docs.streamlit.io/deploy/streamlit-community-cloud